Comandos — ESP32-P4 v1.15.0

Clima externo — 1.15.0

Temperatura e umidade de Porto Alegre vêm do Open-Meteo, nas coordenadas -30.03283, -51.23019, iguais às do M5NanoC6. São condições estimadas pelo serviço meteorológico; a temperatura interna da CPU permanece separada.

A primeira consulta aguarda internet por Wi-Fi ou Ethernet e relógio sincronizado por NTP. Atualiza a cada 10 minutos; em caso de falha, tenta novamente após 2 minutos. O cache expira em 30 minutos, considerando também o horário informado pela API; dados ausentes ou vencidos viram `NaN`. A consulta HTTPS roda em tarefa própria no core 0, com resposta limitada a 2048 bytes. `/metrics` lê somente o cache. Novas consultas são suspensas durante OTA; uma consulta já iniciada pode terminar.

Use `weather_status` pela Serial, UDP ou terminal web para ver rede, NTP, validade, temperatura, umidade, idade e último resultado HTTP. Instale **ArduinoJson 7.x** (revisado com os cabeçalhos 7.4.3); HTTPClient e NetworkClientSecure acompanham o pacote Arduino ESP32.

Revisão documental: **31/10/2026**.

Estado conferido com o código **1.15.0**: GPIO27 para navegação, protetor de robôs, som de troca de página e o comando `reset_counters`. A compilação e a gravação permanecem pendentes; esta revisão também altera o firmware.

Desde 1.13.2, o comando `temp` compartilha o sensor com a métrica `esp_cpu_temperature_celsius` do Prometheus/Grafana. Em falha, responde `Temp: indisponivel`; a métrica fica `NaN`, sem retornar zero falso. CHANGELOG.md e catálogo geral.

Envie por UDP à porta **4210**, pela serial a **115200 baud** ou pelo terminal web em `/info`. O firmware responde pelo canal de origem. Na serial, termine cada comando com LF, CR ou CRLF e aguarde o fim da inicialização. Limite: **254 bytes por comando**. Nomes não distinguem maiúsculas/minúsculas; argumentos preservam a caixa. Arquivos SD podem gerar várias respostas UDP.

Prometheus pela interface web

O padrão de conexão segue o M5NanoC6. Configure endereço do servidor, porta de métricas da placa e intervalo (5–3600 s) em /prometheus, na porta 80. As Preferences preservam esses valores. A porta muda após cerca de um segundo, sem reiniciar; a web e a API permanecem na 80.

Na porta 80, /metrics compartilha a web. Nas outras portas, o servidor de coleta atende apenas /metrics. A porta 3232 é reservada ao OTA. Não há envio ativo. Não há mais chave de ativação ou campo para porta de acesso ao servidor Prometheus.

Endpoint na porta 80	Ação
GET /prometheus	Formulário de conexão e YAML gerado.
GET /api/prometheus/config	host, port, active_port, web_port, interval, mode, scrape_config, persistent_counters_ok.

Endpoint na porta 80	Ação
POST /api/prometheus/config	Salva formulário host, port e interval; exige X-ESP32-Command: 1.
GET /prometheus.yml	Download do YAML para aplicar no servidor.

POST /api/prometheus é um alias que aceita os mesmos três campos do novo formato. GET /metrics na porta 80 retorna 404 quando outra porta de métricas está ativa. As métricas, inclusive as do SD, são descritas em PROMETHEUS.md. O padrão atual da porta de métricas é **80**; valores legados salvos são preservados na migração. O namespace deste projeto é `prometheus`, não `telemetry`. O endereço do servidor é uma referência, não uma restrição de origem. `reset_counters` zera somente os contadores de boot, reinicialização e watchdog nesse namespace, preservando a configuração de coleta.

Diagnóstico de áudio — 1.14.1

Execute audio_test, aguarde cerca de um segundo e consulte audio_status. O status confirma dados enviados ao I2S, não a saída acústica. Escritas parciais/timeout são registradas e a próxima solicitação pode tentar novamente. A preparação inclui cerca de 100 ms de silêncio para o amplificador. Veja README.md para detalhes.

OLED, botão e áudio

Botão GPIO27/GND (INPUT_PULLUP), ativo em LOW: uma pressão acorda e avança uma página; segurar não apaga Wi-Fi nem repete. A rotação continua a cada oito segundos enquanto o protetor está inativo. Após 60 s sem interação, o robô ocupa todo o OLED, sem desligar a tela. Logs e consultas web/Prometheus não reiniciam a inatividade. Trocas manuais e automáticas entre páginas emitem um bip assíncrono pelo ES8311.

Comando	Função
protetor:on	Mostra as carinhas agora.
protetor:off	Acorda e desativa a entrada automática.
protetor:auto	Acorda e reativa a entrada automática.
protetor:status	Consulta sem acordar ou reiniciar a inatividade.
protetor:tempo:60	Tempo de inatividade, de 5 a 3600 s; fica em RAM.
audio_status	Estado do ES8311/I2S, pinos, erro, contadores, bytes enviados e leitura dos registradores.
audio_test	Solicita teste de 500 ms / 880 Hz sem trocar a página; não recebe argumentos.
botao_status	Pino configurado e leitura LOW/HIGH.

tela:next e tela:auto também saem das carinhas. O terminal web oferece o atalho Carinhas de robô. A configuração detalhada de som está em README.md. Não há handler sd_test nesta versão.

Exemplos para conferir após gravar o firmware:

```
botao_status
audio_test
audio_status
protetor:on
tela:next
protetor:tempo:60
protetor:auto
```

`botao_status` deve indicar GPIO27 e LOW ao pressionar. `audio_status` mostra se o codec/I2S ficou pronto e o último erro. `protetor:on` permite observar as carinhas sem aguardar o tempo de inatividade; `tela:next` acorda e avança. Os ajustes do protetor ficam em RAM: ao reiniciar, volta a entrada automática após 60 s. O estado das métricas continua independente desses ajustes.

Sistema

Comando	Função	Exemplo
`help`	Lista comandos e aliases.	`help`
`name`	Identifica o dispositivo; responde `ESP32P4`.	`name`
`info`	Informações gerais da placa.	`info`
`status`	Resumo de Wi-Fi, SD, relógio e heap.	`status`
`version`	Versão do firmware.	`version`
`build`	Data e hora da compilação.	`build`
`crash_info`	Lê o crash, arquiva e verifica no SD, e só então apaga o registro interno.	`crash_info`
`reason`	Motivo do último reset.	`reason`
`alive`	Responde ALIVE.	`alive`
`cores`	Core de execução, tarefa serial e ocupação da fila.	`cores`
`ota_info`	Serviço OTA, porta e tamanho da partição.	`ota_info`
`ota_password`	Mostra usuário `admin` e senha OTA fixa `123456` somente na serial local.	`ota_password`
`uptime`	Tempo ligado em milissegundos.	`uptime`
`lastcmd`	Último comando; credenciais ficam ocultas.	`lastcmd`
`cmdcount`	Total de comandos, exceto lastcmd/cmdcount.	`cmdcount`
`reboot`	Reinicia após responder.	`reboot`
`reset_counters`	Zera somente os contadores NVS de boot/reboot/watchdog.	`reset_counters`

Diagnóstico e hardware

Comando	Função	Exemplo
`health`	Diagnóstico de rede, OLED, SD montado, relógio e memória. Somente leitura.	`health`
`selftest`	Mesmo diagnóstico de health; não realiza teste de escrita ou teste elétrico.	`selftest`
`temp`	Temperatura interna do processador.	`temp`
`cpu`	Modelo, revisão e frequência.	`cpu`
`chip_info`	Modelo, revisão, núcleos, frequência e SDK.	`chip_info`
`ram`	Heap livre, mínimo e maior bloco disponível.	`ram`
`heap`	Heap livre em bytes.	`heap`
`heap_min`	Menor heap livre desde o boot.	`heap_min`
`psram`	PSRAM total e livre em bytes; zero quando indisponível.	`psram`
`flash`	Flash total, tamanho do sketch e espaço de sketch livre.	`flash`

Rede

Comando	Função	Exemplo
`net_info`	IP, gateway, máscara e DNS de Wi-Fi e Ethernet, com estado das interfaces.	`net_info`
`net_monitor`	RTT atual/mínimo/médio/máximo, idade da medição, quedas, tempo offline e eventos.	`net_monitor`
`eth_status`	Estado do cabo, conexão Ethernet, IP, gateway e MAC.	`eth_status`
`wifi_status`	Estado Wi-Fi, código numérico de status/modo, SSID e IP STA/AP.	`wifi_status`
`channel`	Canal Wi-Fi.	`channel`
`mac`	MAC Wi-Fi.	`mac`
`rssi`	RSSI Wi-Fi em dBm.	`rssi`
`ip`	Endereços Wi-Fi e Ethernet.	`ip`
`ssid`	SSID conectado.	`ssid`
`wifi_list`	Somente SSIDs salvos, um por linha; lista vazia responde uma linha vazia.	`wifi_list`
`wifi_add:SSID|SENHA`	Salva/atualiza uma rede; senha vazia cadastra rede aberta. Não reinicia.	`wifi_add:MinhaRede|MinhaSenha123`

Comando	Função	Exemplo
`reset_wifi`	Apaga todas as redes salvas e reinicia.	`reset_wifi`

OLED

Comando	Função	Exemplo
`tela:0`	Relógio e data.	`tela:0`
`tela:1`	Wi-Fi: rede, IP, gateway, RSSI e canal.	`tela:1`
`tela:2`	Ethernet: IP, gateway, quedas e tempo offline.	`tela:2`
`tela:3`	Sistema e SD.	`tela:3`
`tela:4`	Ping aos gateways e gráfico.	`tela:4`
`tela:5`	Últimos quatro eventos de conexão.	`tela:5`
`tela:next`	Avança e mantém rotação automática.	`tela:next`
`tela:auto`	Retoma rotação a cada oito segundos.	`tela:auto`

Horário

Comando	Função	Exemplo
`time`	Hora local.	`time`
`date`	Data local.	`date`
`ntp_status`	Validade do relógio, data/hora e servidores configurados; não comprova alcance atual do NTP.	`ntp_status`
`set_fuso:N`	Salva offset UTC inteiro de -12 a +14.	`set_fuso:-3`
`dst_on`	Salva horário de verão de uma hora.	`dst_on`
`dst_off`	Desativa e salva horário de verão.	`dst_off`
`dst:0 ou dst:1`	Define horário de verão explicitamente.	`dst:0`

LED

Comando	Função	Exemplo
`led_on`	Acende e cancela sequência de pisca.	`led_on`
`led_off`	Apaga e cancela sequência de pisca.	`led_off`
`led_blink[:ms]`	Pisca durante a duração indicada, alternando a cada 200 ms, sem bloquear o loop. Padrão 3000 ms, máximo 10000 ms.	`led_blink:5000`

Cartão SD

Comando	Função	Exemplo
`sd_status`	Capacidade total, usada e livre.	`sd_status`
`sd_list`	Lista arquivos na raiz; não recebe argumento de pasta nesta versão.	`sd_list`
`sd_read:arquivo`	Envia o arquivo inteiro em múltiplos pacotes UDP.	`sd_read:/dados.txt`
`sd_write:arquivo:texto`	Grava o texto substituindo o conteúdo.	`sd_write:/dados.txt:Primeira linha`
`sd_append:arquivo:texto`	Acrescenta texto e quebra de linha CRLF; cria o arquivo se necessário.	`sd_append:/dados.txt:Outra linha`
`sd_del:arquivo`	Remove arquivo.	`sd_del:/dados.txt`
`sd_exists:arquivo`	Verifica existência.	`sd_exists:/dados.txt`
`sd_size:arquivo`	Tamanho em bytes.	`sd_size:/dados.txt`
`sd_tail:arquivo[:linhas]`	Envia as últimas linhas; padrão 10.	`sd_tail:/dados.txt:5`
`sd_log[:linhas]`	Últimas linhas de /log.txt; padrão 10.	`sd_log:20`
`sd_clear_log`	Apaga /log.txt.	`sd_clear_log`
`sd_mkdir:pasta`	Cria pasta; a pasta pai precisa existir.	`sd_mkdir:/dados`
`sd_rmdir:pasta`	Remove pasta vazia; raiz não é permitida.	`sd_rmdir:/dados`

Aliases compatíveis

Alias	Comando equivalente
`init`	`reason`
`flash_info`	`flash`
`restart`	`reboot`
`list`	`sd_list`
`read`	`sd_read`
`write`	`sd_write`

Alias	Comando equivalente
delete	sd_del
exists	sd_exists
size	sd_size
tail	sd_tail
sd_info	sd_status
set_tz	set_fuso
set_dst	dst

Cadastro circular de redes

```
wifi_add:MinhaRede|MinhaSenha123
wifi_add:RedeAberta|
wifi_list
reboot
```

Novos SSIDs ocupam o próximo perfil circular, começando no perfil 1; após o quinto, a próxima inserção substitui o perfil 1. Atualizar um SSID existente troca a senha sem avançar o cursor. `wifi_add` verifica a gravação e tenta restaurar o perfil anterior se falhar. `reboot` aplica a nova lista na busca de redes.

O cursor UDP é independente da última rede utilizada; o formulário HTTP edita os cinco perfis diretamente e mantém o cursor. O SSID não pode conter `|`; a senha pode conter `|`, pois só o primeiro separador é interpretado. SSID: de 1 a 32 bytes. Senha: de 8 a 63 bytes, 64 caracteres hexadecimais ou vazia para rede aberta. Espaços da senha são preservados; CR/LF finais do comando são removidos. Credenciais não aparecem no terminal, no log SD nem em `lastcmd`.

Operações que alteram dados

`reboot`, `reset_wifi`, `wifi_add`, ajustes de fuso/DST e comandos de escrita, exclusão e pastas SD alteram o dispositivo. O firmware executa diretamente; a confirmação, quando houver, pertence ao aplicativo que envia os comandos. `sd_write` substitui o arquivo, `sd_append` acrescenta uma linha e `sd_clear_log` apaga o log. Operações NVS com vários campos não são atômicas contra falta de energia.

Botão e portal

GPIO27/GND: pressione para acordar e avançar a tela. Segurar não limpa redes. Use reset_wifi para essa operação. `/info` mostra informações; `/wifi` edita os cinco perfis.

Interpretação dos diagnósticos

Ping mede o gateway de cada interface, não o acesso à Internet. Um timeout pode ser bloqueio de ICMP. Estatísticas do monitor ficam em RAM e reiniciam no boot. `ntp_status` consulta a validade do relógio local, sem disparar nova consulta NTP. `health` e `selftest` consultam estados já disponíveis; não testam escrita no

SD nem garantem a integridade física do hardware. Algumas operações SD continuam bloqueantes e podem atrasar telas e monitor.

Serial USB e terminal web (v1.8.0)

Todos os comandos deste documento também funcionam na serial, a **115200 baud**, e no terminal da página `/info`. Na serial, envie uma linha por comando com final LF, CR ou CRLF. Aguarde o fim da inicialização da placa. Limite: **254 bytes**. Para a porta USB nativa, habilite **USB CDC On Boot** conforme a configuração da placa.

As respostas são enviadas ao canal de origem. No terminal web, respostas acima de 8 KB são truncadas com um aviso; use `sd_tail` para consultar arquivos grandes. O terminal pede confirmação para comandos que alteram arquivos/configurações. Se a conexão cair, verifique o estado antes de repetir um comando de escrita.

crash_info — 1.12.0

O comando `crash_info` segue o central: informa o motivo do último reset, lê o core dump válido, arquiva o relatório no SD e só apaga o registro interno após reabrir o arquivo e conferir tamanho e conteúdo byte a byte. Sem SD, com gravação incompleta ou falha na conferência, o registro interno permanece para nova tentativa.

O relatório mostra panic, tarefa, PC e identificador SHA256 do ELF. No ESP32-P4 (RISC-V), inclui RA, SP, MCAUSE, MTVAL e a pilha bruta fornecida pelo SDK; a decodificação do backtrace exige o ELF do firmware que sofreu o crash. Preserve esse arquivo junto de cada versão. Não são exibidos endereços da pilha como se fossem um backtrace já decodificado.

Os arquivos são `/crash_YYYYMMDD_HHMMSS.txt` ou `/crash_SEM_NTP_<uptime>.txt`, com sufixo numérico quando necessário. Use `sd_list` e `sd_read:/crash_....txt` para consultá-los. A página `/logs` continua lendo apenas `/log.txt`.

Disponível pela serial, UDP e terminal web. No UDP, o relatório é dividido em pacotes de até 1200 bytes para caber no receptor do central; a entrega continua sujeita às perdas e à ordem do UDP. No navegador, a operação pede confirmação, pois arquiva dados e apaga o registro interno depois da verificação.

Sem crash válido, informa essa condição e o último reset, sem criar arquivo ou apagar dados. Se o SDK não habilitar core dump em flash/ELF, informa que o recurso está desabilitado. O SDK ESP32-P4 3.3.12 e a tabela padrão usados no projeto já incluem esse suporte e a partição coredump. O comando não provoca um crash.